

Walkie Talkie Mesh Chat

by John Dovey

Walkie Talkie Mesh Chat

By **John Dovey**

Created with eBook Editor 2.32.1 on 2026-07-14 18:10

Copyright © 2026 John Dovey

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law. This is a work of fiction. Names, characters, businesses, places, events, and incidents are either the products of the author's imagination or used in a fictitious manner.

Table of Contents

Title Page	i
Imprint	ii
Chapter 1: Getting Started	1
Chapter 2: Windows Builds	2
Chapter 3: Android Client	3
Chapter 4: macOS builds	11
Chapter 5: Android Client	12
Chapter 6: The iPhone app	20
Chapter 7: The Base Station	22
Chapter 8: MeshBridge	28
Chapter 9: MeshSniff	31
Chapter 10: Wearables	32
About the Author	33
Notes	34

Getting Started

WalkieTalkie is a cross-platform push-to-talk (PTT) app. Later chapters cover the desktop **Base Station** (Mac / Windows / Linux), the Android and iPhone clients, wearables, optional **MeshBridge** for syncing devices and voice notes across separate networks (not live Talk), and **MeshSniff** for a LAN discovery map. Android + Base Station are confirmed working together on real hardware. The iPhone app is in progress (buildable; device mesh verify pending).

Chapter 2: Windows Builds

Windows builds

A Windows Base Station is built by cross-compiling from the Mac dev machine — there's no need for an actual Windows machine to produce the .exe. Run `./tools/build-windows-server.sh` from the repo root; the first run takes a few extra minutes to set up a one-time cross-compile toolchain, after which it just rebuilds the executable. The resulting .exe has full mic/speaker support, not just the registry/web-UI/relay pieces. This hasn't been tested on real Windows hardware yet — only confirmed to be a valid Windows executable with sane-looking dependencies.

MeshBridge for Windows is produced the same way with `./tools/build-windows-meshbridge.sh`. See the MeshBridge chapter for how it runs next to a Base Station.

Chapter 3: Android Client

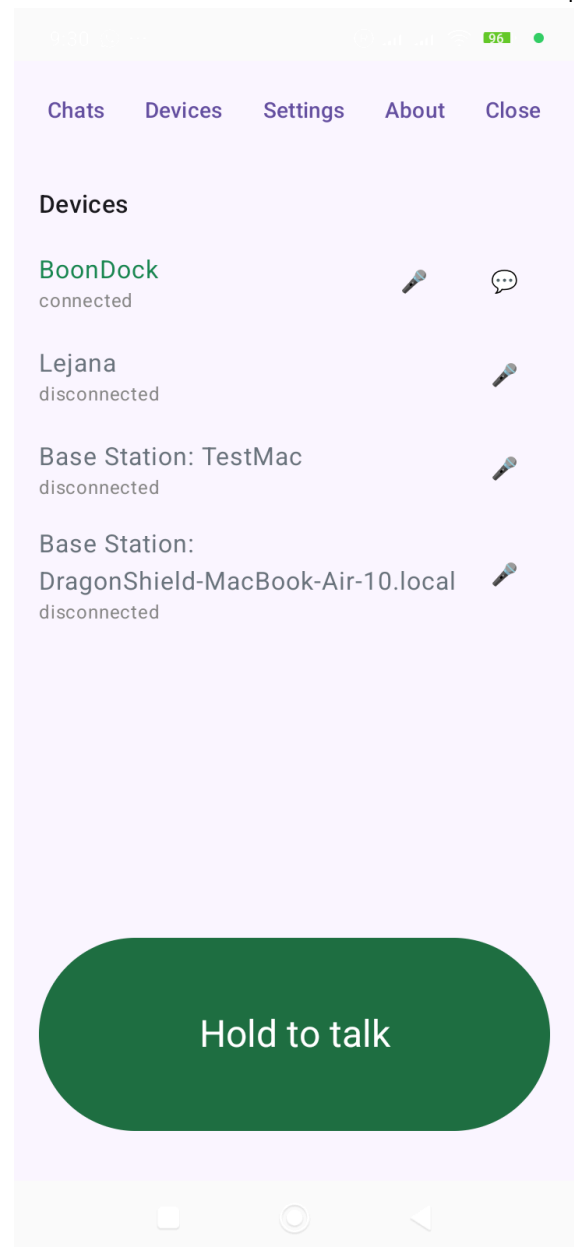
The Android app

The Android app was confirmed working on real hardware for the first time on 2026-07-13: it discovers a Base Station over Wi-Fi, connects to it, and exchanges real two-way audio — confirmed in both directions, including someone pressing the phone's Talk button and hearing their own voice come out of the desktop machine's speakers.

A few things worth knowing if you're trying this yourself:

- The app needs microphone, location, and Bluetooth permissions — grant all of them when asked, or nothing will work.
- On some phones (confirmed on a Xiaomi/HyperOS device), aggressive battery management kills the app's network activity a few seconds after connecting, even though it's a proper foreground service. If audio drops after several seconds, check your phone's battery settings and allow the app to run unrestricted / disable battery optimization for it.

Devices



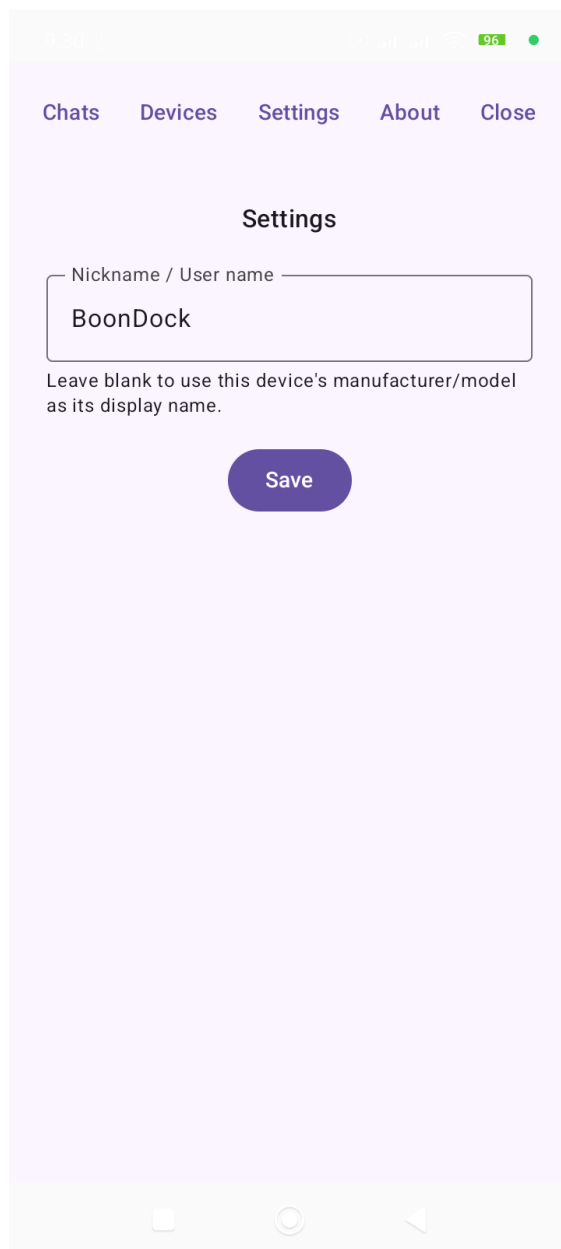
Devices screen — connected and disconnected peers, mic and chat actions, and Hold to talk

The home screen is **Devices**: a list of peers seen on the mesh, per-device actions, and the green **Hold to talk** button for group push-to-talk.

Connected peers appear in green. Each row can offer a microphone control (start a voice message) and, when the peer is connected, a chat control (invite a private channel). The Devices list is sorted most-recently-seen first, and — unlike the desktop Base Station, which keeps every device it's ever seen — the phone automatically forgets (deletes) any device it hasn't seen in over 48 hours, to keep the on-device list from accumulating clutter over time.

Settings

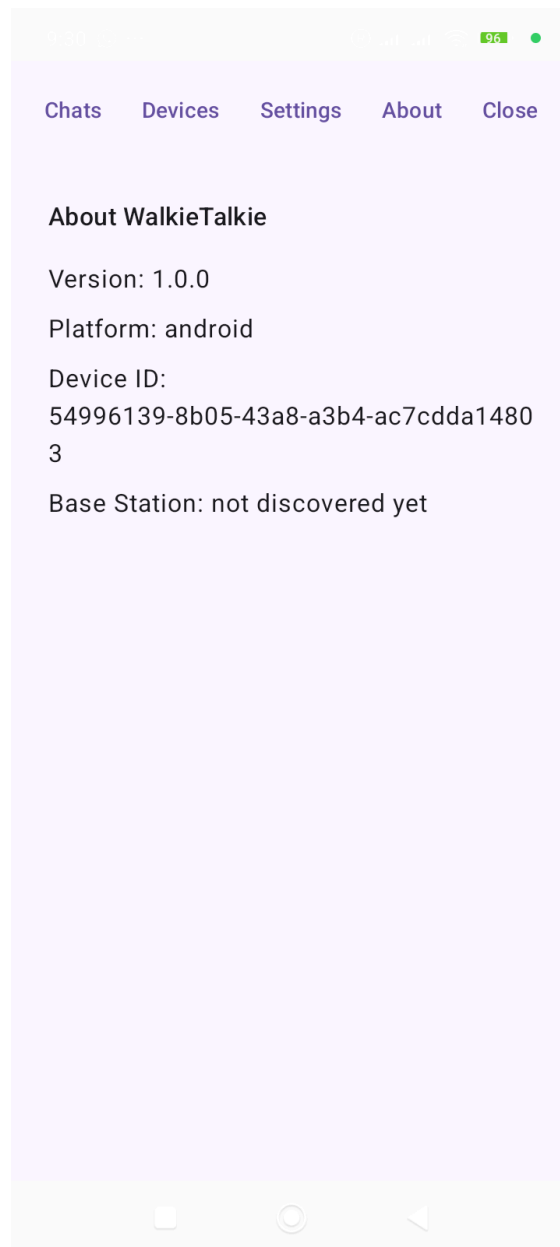
Use **Settings** to set a Nickname / User name for this device. Leave it blank to fall back to the phone's manufacturer and model as the display name.



Settings — nickname field and Save

About

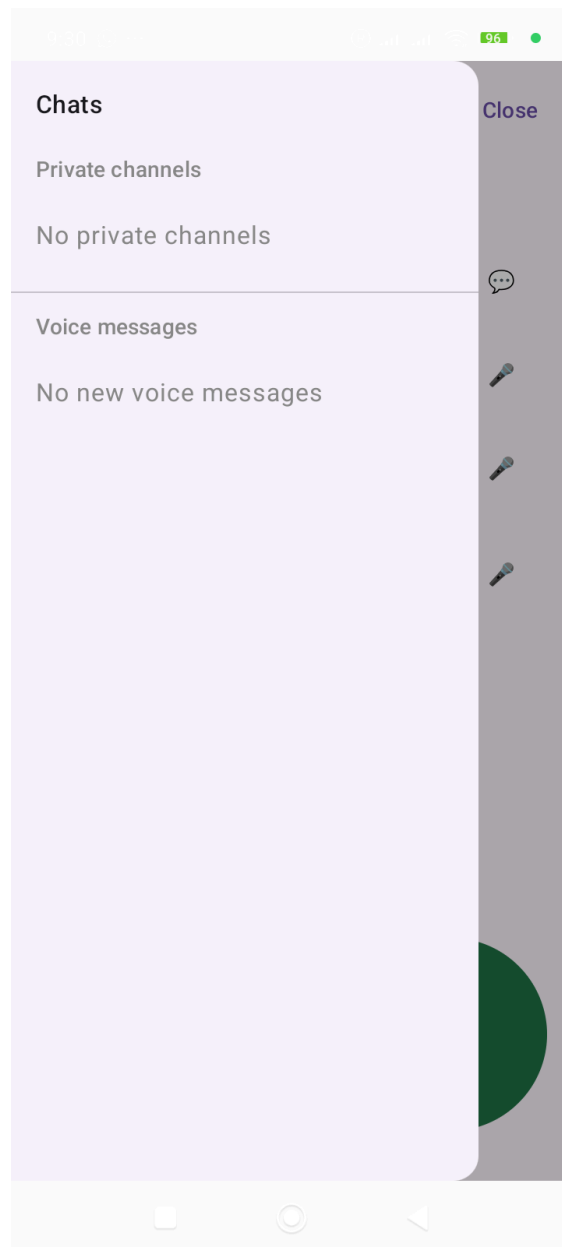
About shows this app's version, platform, this phone's device ID, and the discovered Base Station URL (or “not discovered yet”).



About WalkieTalkie — version, device ID, and Base Station status

Chats drawer

The top **Chats** control opens a side drawer for private channels and queued voice messages. Unread counts appear as red badges when there is something waiting.

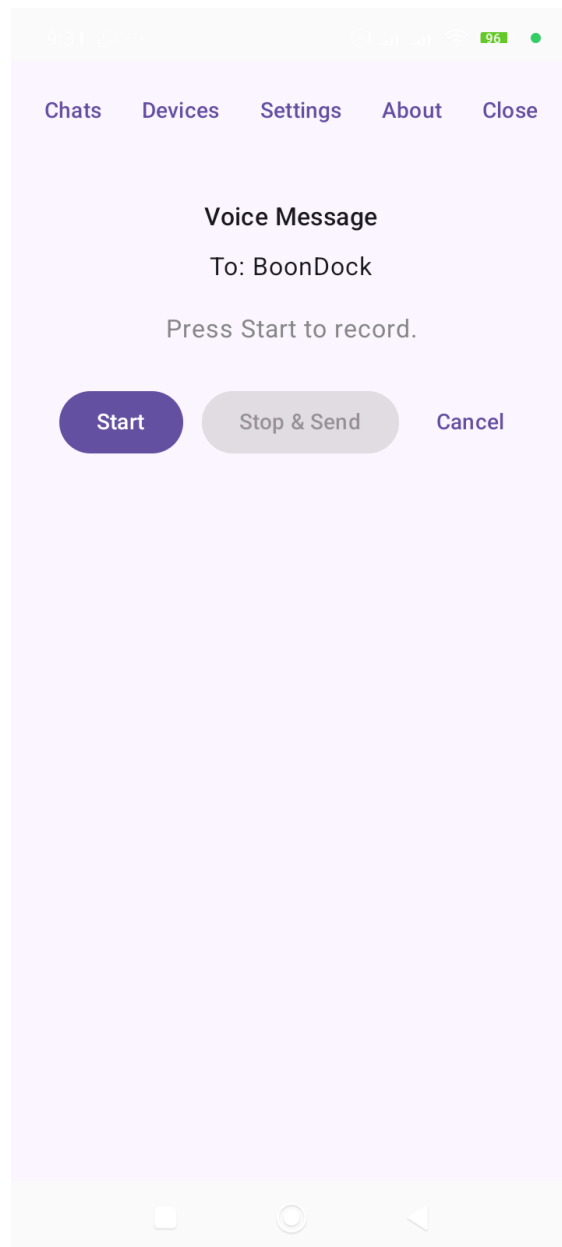


Chats drawer — private channels and voice messages (empty state)

Voice messages

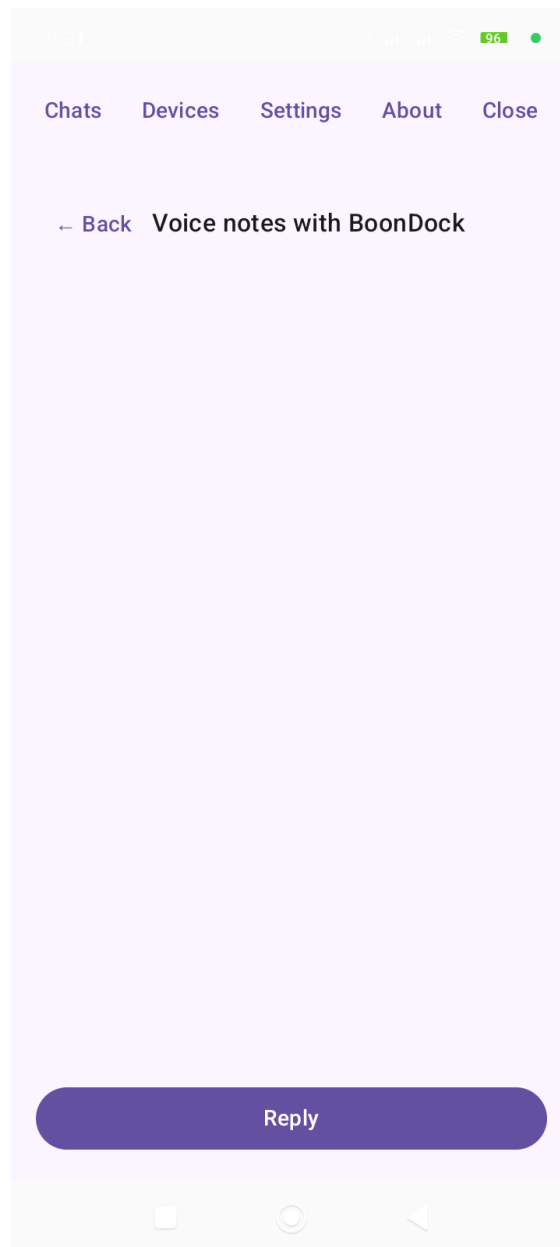
Voice messages and private channels need a Base Station on the LAN; without one, group Hold-to-talk still works, but voice-message and private-chat actions will fail until a Base Station appears.

From a device row, open the microphone action to record a clip for that peer.



Voice Message — Start, Stop & Send, or Cancel a clip to a peer

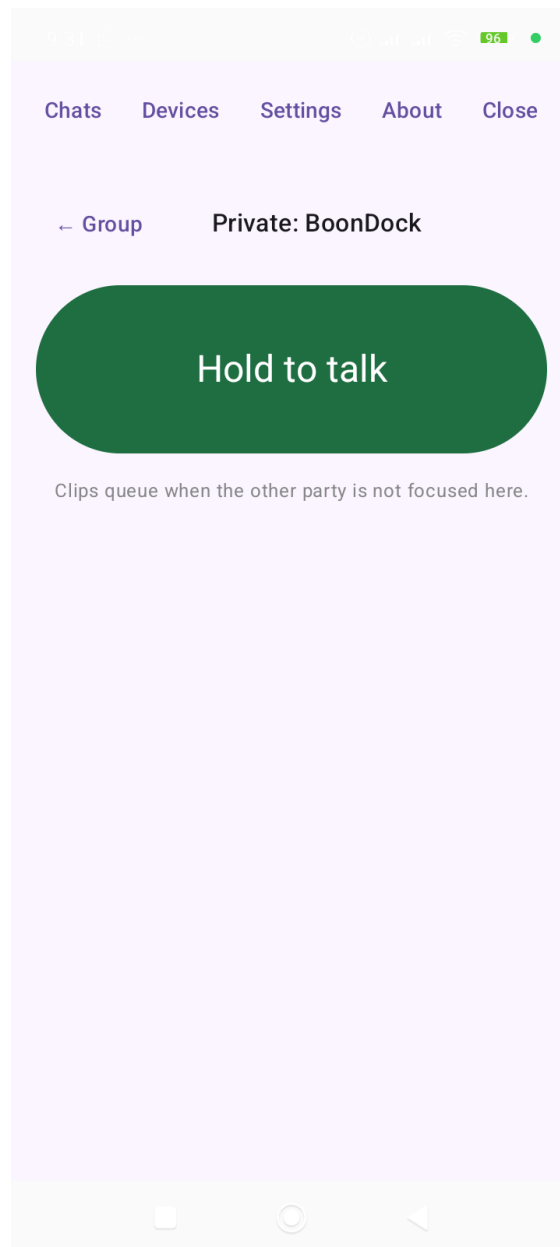
The voice-notes thread lists clips with a peer; play or delete them, or tap **Reply** to record another.



Voice notes thread with a peer — list and Reply

Private channels

When a peer is connected, the chat action invites a private channel (extra peers can be invited later). That screen has its own Hold to talk control — live over the mesh or SFU room when peers are reachable, otherwise clips that queue if they are not focused on the channel.



Private channel — Hold to talk for a 1:1 clip channel

Not yet tested: whether a third device can detect the phone over Bluetooth and report its presence to a Base Station when the phone has no Wi-Fi — that needs a second Bluetooth-capable device to verify.

Chapter 4: macOS builds

macOS builds

A macOS Base Station binary for **both** Apple Silicon and Intel is produced by `./tools/build-macos-server.sh` from the repo root. The first run builds a per-architecture Opus toolchain (cached afterward). It writes three files under `/Volumes/JohnDovey/tmp` by default: `walkietalkie-server-darwin-arm64`, `walkietalkie-server-darwin-amd64`, and a universal `walkietalkie-server-darwin-universal` that runs on either chip. All include full mic/speaker support.

The optional **MeshBridge** companion (see its Manual chapter) is built with `./tools/build-macos-meshbridge.sh` into the same tmp area. Run it alongside a Base Station when you need dual-LAN or punch sync.

Chapter 5: Android Client

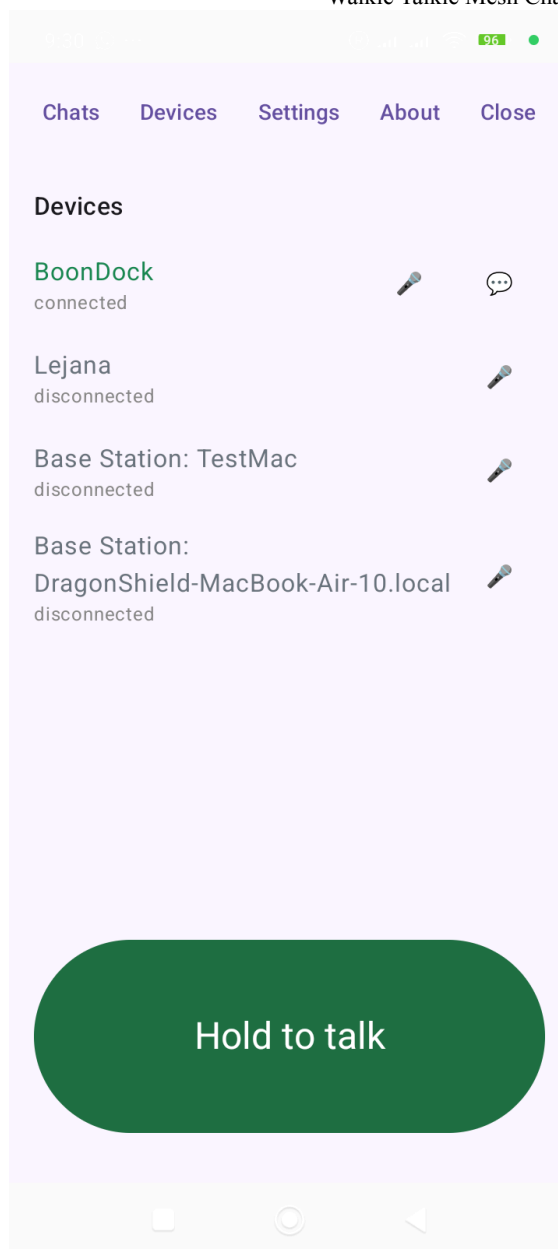
The Android app

The Android app was confirmed working on real hardware for the first time on 2026-07-13: it discovers a Base Station over Wi-Fi, connects to it, and exchanges real two-way audio — confirmed in both directions, including someone pressing the phone's Talk button and hearing their own voice come out of the desktop machine's speakers.

A few things worth knowing if you're trying this yourself:

- The app needs microphone, location, and Bluetooth permissions — grant all of them when asked, or nothing will work.
- On some phones (confirmed on a Xiaomi/HyperOS device), aggressive battery management kills the app's network activity a few seconds after connecting, even though it's a proper foreground service. If audio drops after several seconds, check your phone's battery settings and allow the app to run unrestricted / disable battery optimization for it.

Devices

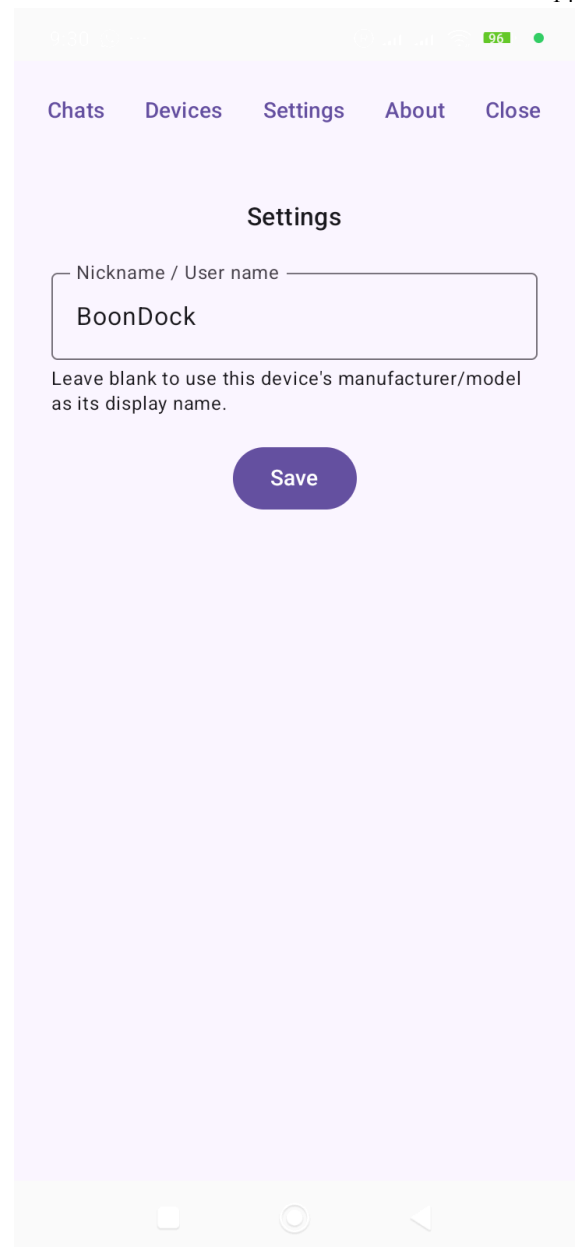


Devices screen — connected and disconnected peers, mic and chat actions, and Hold to talk

The home screen is **Devices**: a list of peers seen on the mesh, per-device actions, and the green **Hold to talk** button for group push-to-talk.

Connected peers appear in green. Each row can offer a microphone control (start a voice message) and, when the peer is connected, a chat control (invite a private channel). The Devices list is sorted most-recently-seen first, and — unlike the desktop Base Station, which keeps every device it's ever seen — the phone automatically forgets (deletes) any device it hasn't seen in over 48 hours, to keep the on-device list from accumulating clutter over time.

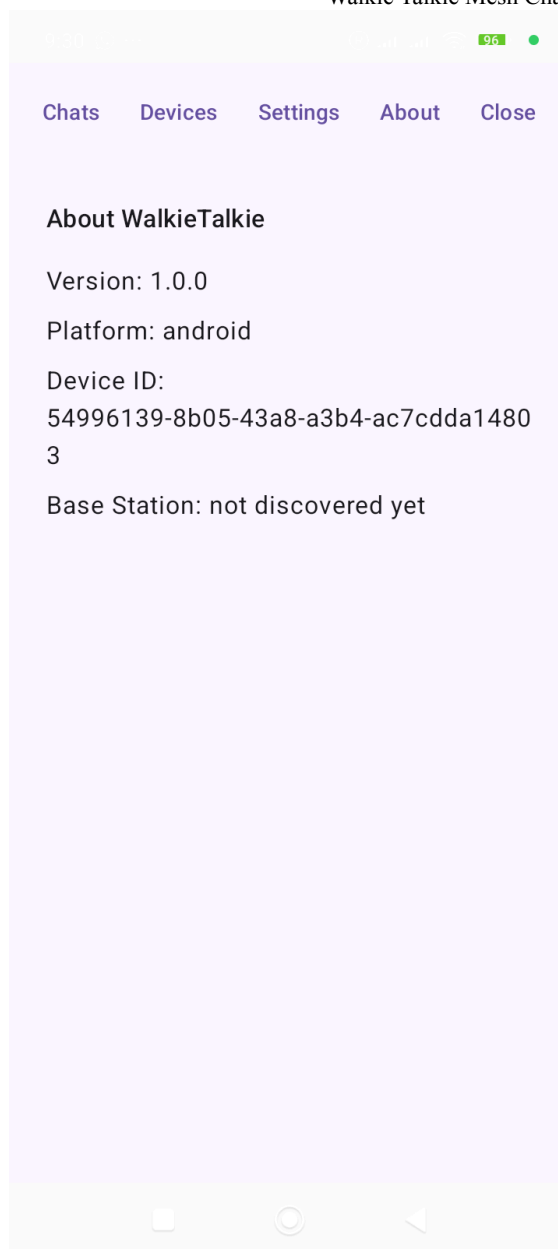
Settings



Settings — nickname field and Save

Use **Settings** to set a Nickname / User name for this device. Leave it blank to fall back to the phone's manufacturer and model as the display name.

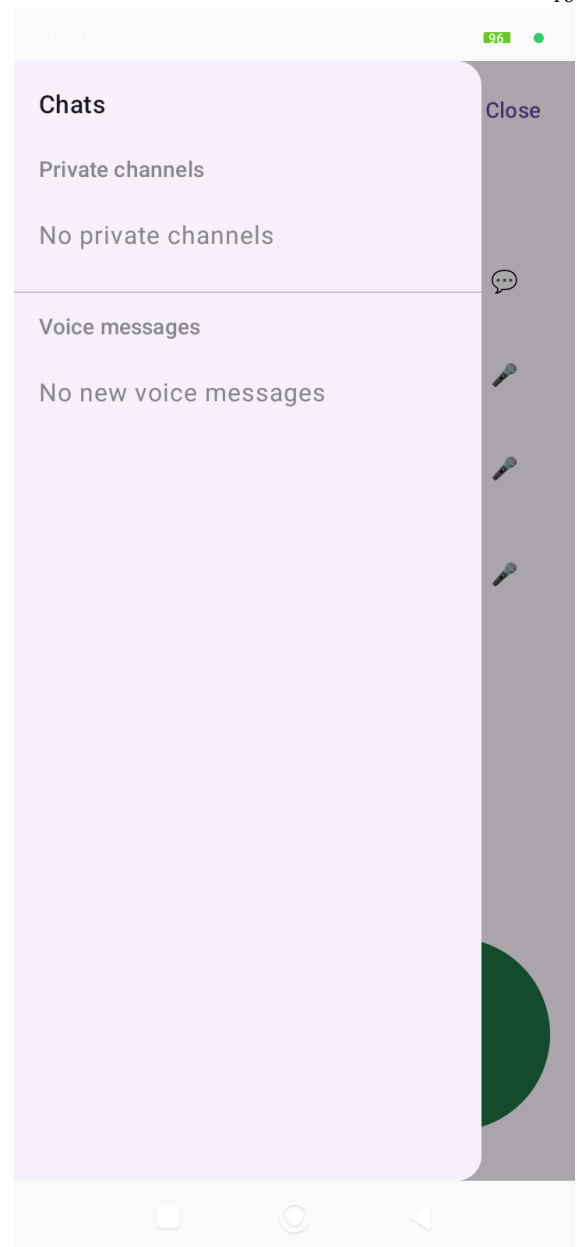
About



About WalkieTalkie — version, device ID, and Base Station status

About shows this app's version, platform, this phone's device ID, and the discovered Base Station URL (or “not discovered yet”).

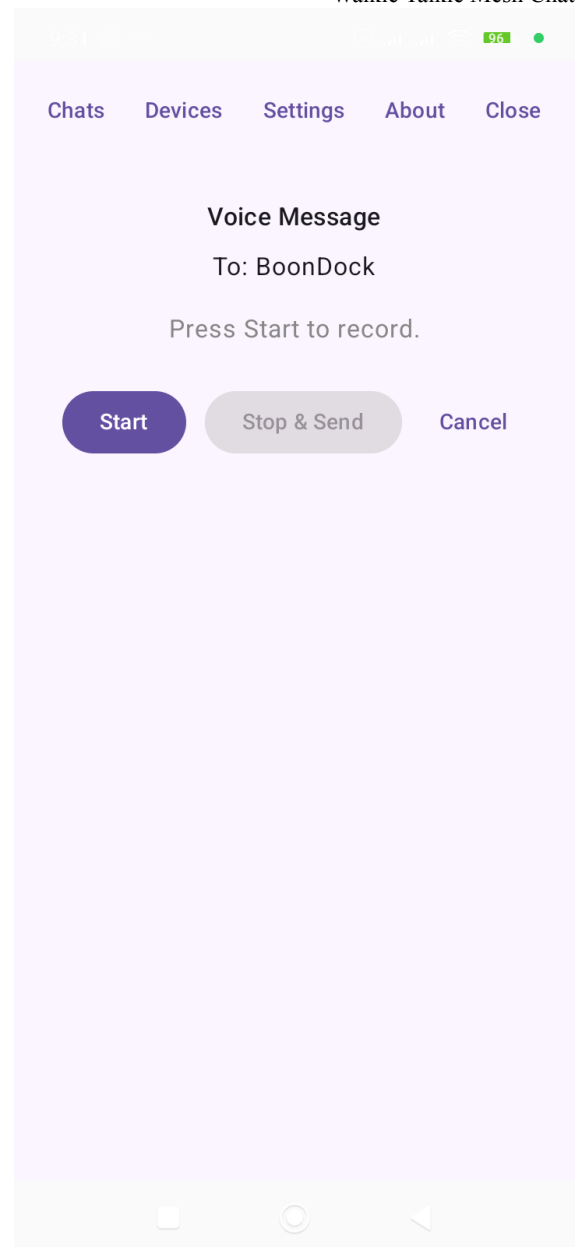
Chats drawer



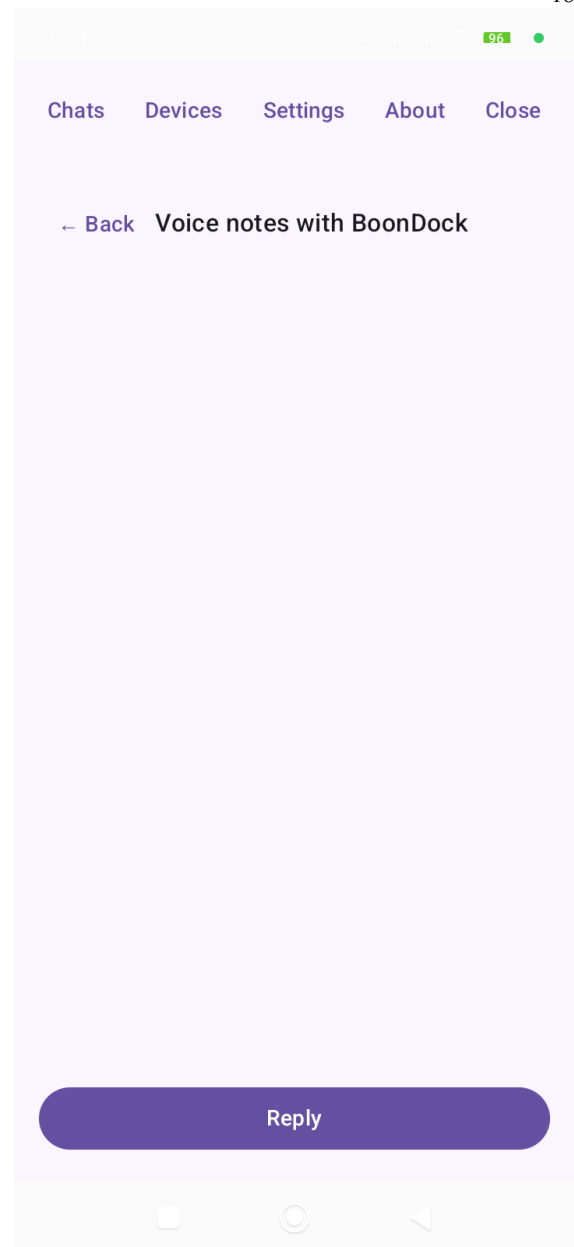
Chats drawer — private channels and voice messages (empty state)

The top **Chats** control opens a side drawer for private channels and queued voice messages. Unread counts appear as red badges when there is something waiting.

Voice messages



Voice Message — Start, Stop & Send, or Cancel a clip to a peer



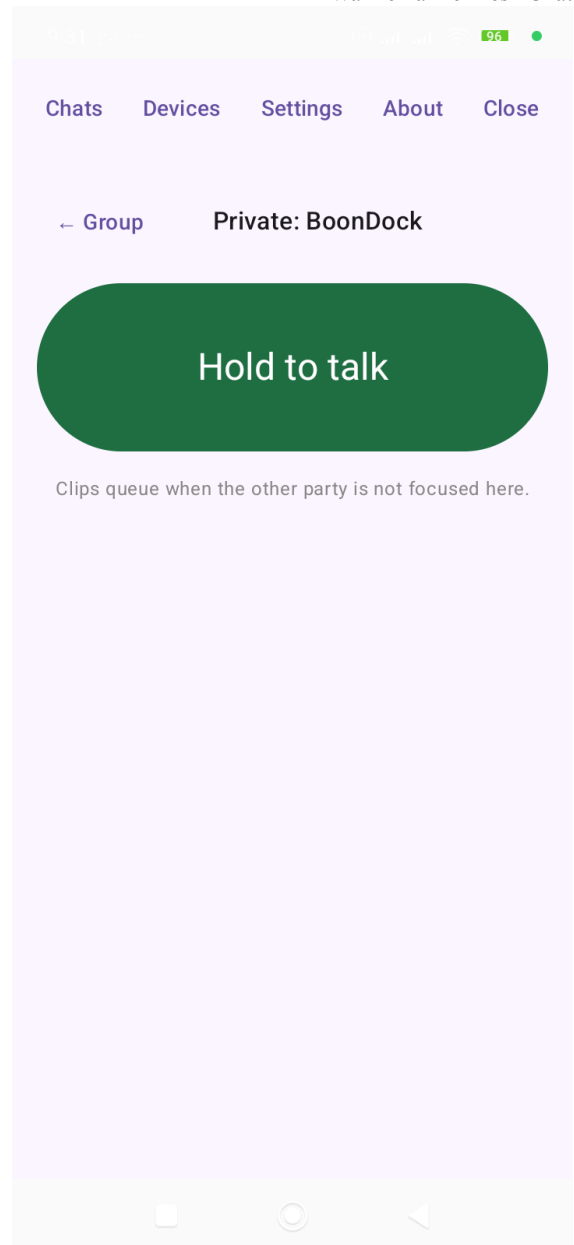
Voice notes thread with a peer — list and Reply

Voice messages and private channels need a Base Station on the LAN; without one, group Hold-to-talk still works, but voice-message and private-chat actions will fail until a Base Station appears.

From a device row, open the microphone action to record a clip for that peer.

The voice-notes thread lists clips with a peer; play or delete them, or tap **Reply** to record another.

Private channels



Private channel — Hold to talk for a 1:1 clip channel

When a peer is connected, the chat action invites a private channel. Channels can include more than two people once extra peers are invited and accept. Hold to talk is **live** when peers share a mesh link or Base Station SFU room; otherwise it records a clip via the Base Station that queues until they focus the channel. The screen shows the current mode (live mesh, live relay, or clip) and lists channel peers by name.

Not yet tested

Whether a third device can detect the phone over Bluetooth and report its presence to a Base Station when the phone has no Wi-Fi — that needs a second Bluetooth-capable device to verify.

The iPhone app

The iPhone app shares the same Go mesh core as Android. Current version is **0.9.0** (voice messages, N-party private channels with live-or-clip Hold to Talk, Hub rooms / SFU notes, MeshSniff /sniff identify, and permission gating).

Building and opening

Before opening the Xcode project, regenerate the gitignored frameworks (same idea as the Android AAR):

```
./tools/gomobile-bind-ios.sh
./tools/build-opus-ios.sh
cd ios && xcodegen generate
# or: ./tools/ios-build.sh
```

Signing: copy ios/Config/Local.xcconfig.example to ios/Config/Local.xcconfig (gitignored) and set DEVELOPMENT_TEAM. Automatic signing. Locked-screen Push to Talk needs a paid Apple Developer team; without it, Simulator / unsigned builds still compile and foreground **Hold to Talk** works.

Permissions

On first launch the app asks for **microphone**, **location**, and **Bluetooth**. Grant mic at least, or Talk will not start. Location feeds the Base Station map; Bluetooth is used for nearby presence when you are not on the same Wi-Fi.

Using the app

- **Devices** — peer list (connected names in green), per-device **VM** (voice message) and **Chat** (private channel invite when connected), plus foreground **Hold to Talk**.
- **Chats** — drawer of private channels (with unread badges) and senders who have queued voice messages.
- **Settings** — nickname (blank falls back to the device name).
- **About** — app version, device ID, discovered Base Station URL.

Voice messages and private-channel clips are Ogg Opus store-and-forward through a LAN Base Station (same APIs as Android). Without a Base Station, group Hold-to-talk still works; VM/Chat will fail until one appears.

Locked-screen PTT uses Apple's **PushToTalk** framework (PTChannelManager). Prefer that on a signed device build; Hold to Talk remains the development fallback.

Battery and Wi-Fi

iOS is generally less aggressive than HyperOS about killing foreground audio, but Low Power Mode /

Background App Refresh can still pause mesh activity if the app is fully backgrounded without an active PTT session — keep the app open while testing LAN Talk. After Wi-Fi drops and returns, the app restarts its mesh node (same practical fix as Android) so mDNS discovery comes back without a full reopen.

See also [docs/2026-07-14-ios-phase4.md](#) for bind commands and architecture notes.

Chapter 7: The Base Station

Desktop

What a Base Station is

Every desktop instance of WalkieTalkie is called a **Base Station**. A Base Station is not just a monitoring dashboard — it's a full participant: it can talk and listen like any other device, and it also hosts a web dashboard showing every device it has seen on the network.

Each Base Station names itself automatically, as "**Base Station: <your computer's name>**" — for example, "Base Station: Johns-MacBook-Air". There's currently no way to rename it from the dashboard; this is intentional (see Settings below).

Running a Base Station

From the server/ directory of the project:

```
go run .
```

This starts the Base Station listening on port **9091** by default. Open <http://localhost:9091> in a browser to see the dashboard. On desktop builds a small **WT** system-tray menu offers **Open Dashboard** and **Quit**; use **-no-tray** for headless / SSH / CI runs.

There is no login. Anyone who can reach the port — anyone on the same LAN, by default — can view the dashboard, including every device's live GPS location if it has one. Don't expose this port beyond a network you trust.

Talking from a Base Station

The big green **Hold to Talk** button at the top of the dashboard's home page lets a person sitting at (or on the same LAN as) the Base Station machine actually use its mic and speaker — press and hold to transmit, release to stop, exactly like the on-screen button in the Android app.

The device list

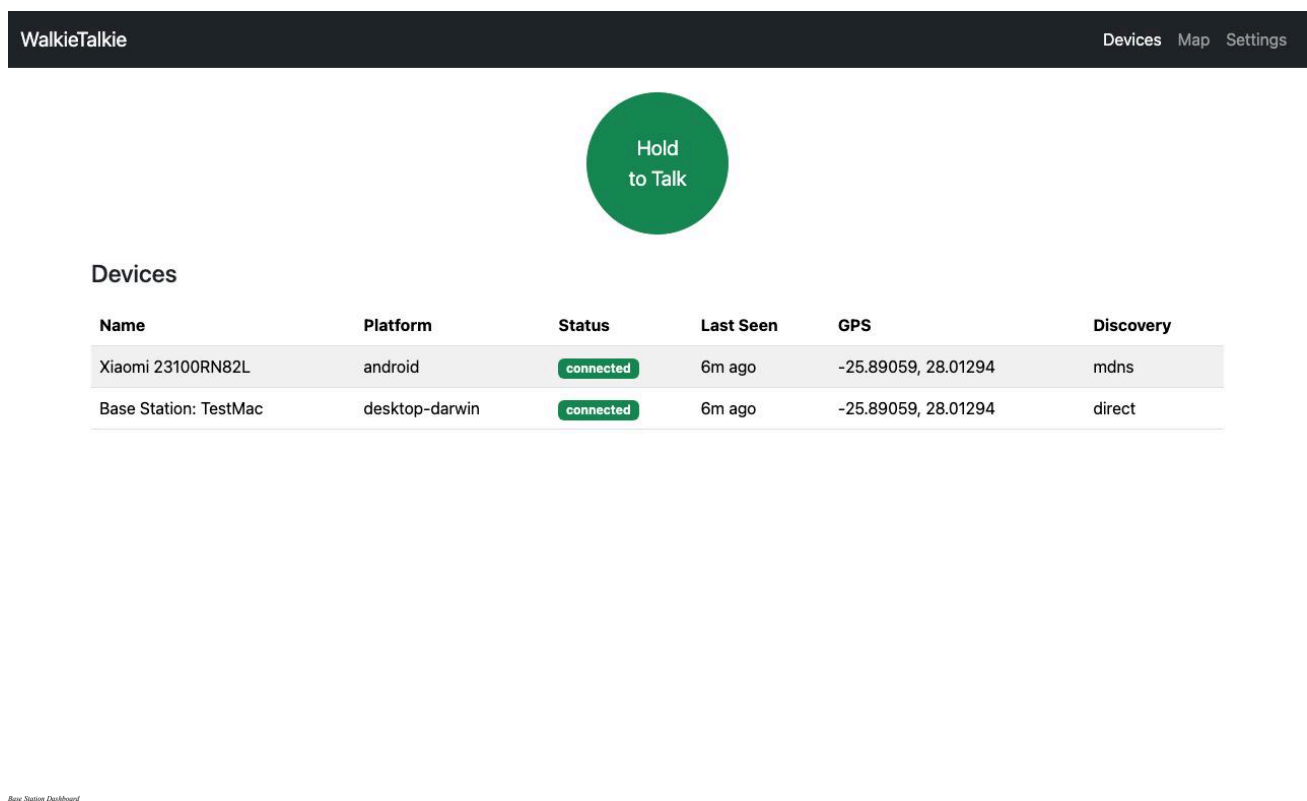
The dashboard's home page (/) lists every device the Base Station has seen recently: its name, platform, connection status (Connected/Disconnected), how long ago it was last seen, its GPS location if known, and how it was discovered (directly, over the LAN via mDNS, or reported by another device over Bluetooth). The list refreshes automatically every few seconds, sorted most-recently-seen first, and each device's name is

colored green if it's currently connected or gray if it's disconnected.

Next to each device are two small action icons: **Voice Message** (always available) opens a recorder so you can send an async voice note that the Base Station holds until that device connects again (notes are kept for at least 21 days, then deleted). **Invite to Chat** appears only for connected devices and starts a private 1:1 channel. If a device has voice notes waiting for it, the list shows how many.

The **Chats** button on the left of the nav opens a drawer with active private channels (and unread counts) and senders who have new voice messages for you. Opening a voice-message entry shows the thread — play, reply, or delete. Opening a private channel switches the Talk control to that pair: when the peer has a direct Wi-Fi mesh link or is on this Base Station's SFU, Hold to Talk is **live unicast**; otherwise it records a clip that queues on this Base Station until they focus the channel. The private panel shows the current mode (live mesh, live relay, or clip).

When two Base Stations are running on the same LAN, they discover each other the same way any two devices do — automatically, with no setup — and each will list the other, including itself as a normal entry in its own list. They also keep their device lists in sync with each other continuously in the background (see "Multiple Base Stations" below).



This is a real screenshot from an actual test session: a Base Station running on a Mac, talking to a real Android phone over Wi-Fi.

Remote Users

The **Remote Users** page (`/remote-users`) lists devices synced from *other* Base Stations by the optional **MeshBridge** companion — across separate Wi-Fi routers or via punch — grouped by their origin Base. Live

Talk does not cross that bridge; voice notes and private-channel clips do. Same-LAN Multi-Base sync (below) already merges peer lists without MeshBridge. See the MeshBridge chapter for setup. For a live topology of every LAN host (routers, phones, Mesh IDs), run the **MeshSniff** companion.

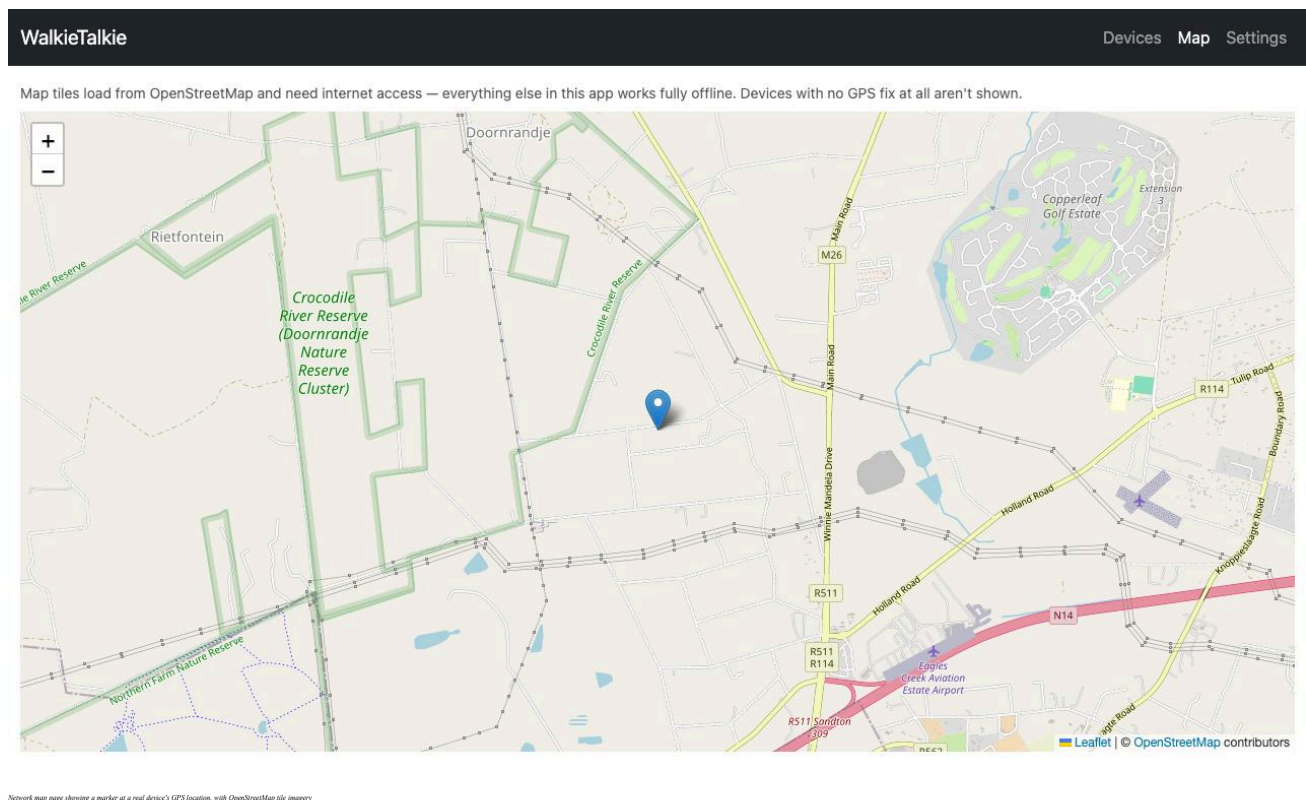
Old Nodes

Any device not seen in over 2 hours moves off the main device list onto its own **Old Nodes** page (/old-nodes) instead of cluttering up the main page or disappearing entirely — this is where a device that crashed, went out of range, or was leftover test data ends up, still sorted most-recently-seen first. The moment that device is seen again, it moves straight back to the main page on its own; nothing needs to be done manually.

The map

The **Map** page (/map) plots every device with a known GPS position. It needs internet access to load the map imagery itself (from OpenStreetMap) — this is the one page in the app that isn't fully offline; everything else, including the device list and Talk button, works with no internet connection at all.

A Base Station machine usually has no GPS hardware of its own. Instead, its position on the map is *estimated* as the average location of every device currently connected to it that has a GPS fix — so it'll only show up once at least one connected device is reporting its location.



Multiple Base Stations

If you run more than one Base Station on the same LAN, they don't just discover each other — they continuously synchronize their device lists with each other in the background, so a device seen by only one Base Station still shows up correctly on the others within a few seconds. If the two Base Stations ever disagree about a device (e.g. its last-seen time or GPS), whichever one saw it most recently wins — with a small clock-skew floor and peer time correction so near-equal clocks don't flap. The same sync also copies private channels and waiting voice-note clips between Base Stations, so a recipient can pick up notes from whichever Base Station they reach.

Each Base Station that receives a GPS fix also keeps a short **local trail** of recent positions for that device (about a week, capped). The API is `GET /api/devices/{id}/gps-history`; trail history is not replicated between Base Stations in this release.

Settings

The **Settings** page (`/settings`) lets you change the Base Station's own configuration: the port it listens on, how often GPS updates are expected, how often it re-syncs with other Base Stations, the relay threshold, whether the relay fallback is enabled, and the **sync clock skew floor** (how many seconds ahead a peer's last-seen must be before it wins a Multi-Base merge). Changing the port restarts the web server on the new port — the page tells you the new address to reconnect to.

The **relay threshold** (10 by default) controls when audio switches from direct device-to-device WebRTC to routing through this Base Station's SFU instead, so a large mesh does not stay fully peer-to-peer. When **Relay enabled** is on and the number of connected peers is already at or above the threshold, *new* connections join via the SFU (also used if a direct ICE attempt fails). Turn Relay enabled off to stay P2P-only. Private channels also use Hub rooms on this SFU for live Talk and, when needed, for voice-note DataChannel forwarding.

The **"Mark disconnected after"** setting (180 seconds by default) controls how long a device can go unseen before it's automatically marked Disconnected, even if it never sent a proper disconnect notice — this is what clears out a device that crashed, walked out of range, or was leftover test data, instead of leaving it stuck showing "connected" forever.

Fixed a related bug where a device's GPS position, name, and app version could get stuck at whatever they were the first time it was discovered, never updating again for the rest of the Base Station's run — even though the device kept sending fresh updates. Both this and the "connected forever" issue above were tracked down after noticing a leftover test entry that wouldn't go away and a phone's map position that never moved.

Settings does **not** let you rename a device from the browser. Every device — including a Base Station — sets its own name locally; the dashboard only ever displays what a device has told it about itself.

WalkieTalkie
Devices
Map
Settings

Settings

Port

Changing this restarts the web server on the new port.

GPS update interval (seconds)

Base Station sync interval (seconds)

How often this Base Station re-syncs its device list with other Base Stations it finds on the LAN.

Relay threshold (connected peers)

Once this many peers are connected at once, route all audio through the relay instead of direct connections.

☒ Enable relay fallback for unreachable peers

Save

Settings page showing port, GPS interval, sync interval, relay threshold, and relay toggle fields

The **Stats** page (/stats) shows usage and traffic for this Base Station: nodes newly seen, private-channel invites/accepts, voice notes and channel clips (counts and bytes), and live group-PTT bytes sent/received by this machine. Ranges: today, week, month, or all time. Peer-to-peer audio that never passes through this Base Station is not included.

About

The **About** page (/about) shows this Base Station's own name, platform, version, and device ID. The device list on the home page also shows the app version each connected device announced itself with, so a mismatched-version device is easy to spot at a glance.

Voice messages and private channels

Both the Base Station dashboard and the phone apps support **async voice messages** and **invite-only private channels** that can grow beyond two people. Clips prefer a direct mesh DataChannel when peers are connected; next they can ride the Base Station SFU DataChannel; otherwise they store-and-forward over HTTP. Multiple Base Stations replicate those inboxes with each other.

- On each device row, a microphone icon sends a **Voice Message** to that node (works even if they are offline — the note is held until they return, for at least 21 days, then deleted). A chat icon **Invite to Chat** appears only while the peer is connected.
- Once a channel is open, you can invite additional connected peers; they accept before joining the participant list. Channel clips fan out to every other member.
- A left-hand drawer lists active private channels (with peer names) and nodes with new voice messages waiting, with count badges.
- Opening a voice-message badge shows the thread for that sender: play, reply, or delete notes.

- Private-channel Hold-to-talk is live when peers are on the mesh/SFU room; otherwise it records a clip scoped to that channel.
- The device list shows how many voice notes are waiting for each node.

Voice notes require a reachable Base Station for the HTTP fallback path. If none is discovered on the LAN and no Direct/SFU path is available, send fails with a clear error rather than dropping silently.

Running more than one instance (for testing)

The desktop app accepts a few flags useful when testing with more than one instance on a single machine:

- `--port`
- `--data-dir`
- `--name`
- `--no-audio` (skip mic/speaker setup, so two instances on one machine don't fight over the same audio hardware)

MeshBridge

MeshBridge is a companion process that runs next to a Base Station. It syncs **devices** and **voice notes** / **private channels** from other Base Stations — across separate Wi-Fi routers, or (with punch) across the internet — into this Base Station's **Remote Users** tab.

It does **not** bridge live Talk (group PTT or private live unicast). Phones and Wear stay on their own LAN; only registry visibility and store-and-forward clips cross the bridge.

When to use it

- Two offices / homes on different Wi-Fi routers, and you want each Base Station to *see* users and voice notes from the other.
- One Mac that can reach both networks at once (Wi-Fi on LAN A + Ethernet into router B).
- Bases that cannot discover each other on a shared LAN (CGNAT / remote sites) via QuakeMesh-style UDP punch and an optional hub.

Same-LAN Multi-Base sync (already covered in The Base Station) does **not** need MeshBridge.

Running MeshBridge

Start the local Base Station first (default `http://127.0.0.1:9091`), then:

```
source tools/go-env.sh
go run ./meshbridge/cmd/meshbridge
```

Or run a release binary from `./tools/build-macos-meshbridge.sh` (also `build-windows-meshbridge.sh` / `build-linux-meshbridge.sh`).

MeshBridge prints its status URL (default **`http://127.0.0.1:9095`**). Settings live in:

- macOS: `~/Library/Application Support/WalkieTalkie/meshbridge/settings.json`
- Linux: `~/.config/WalkieTalkie/meshbridge/settings.json`
- Windows: under the user AppData config directory for WalkieTalkie/meshbridge

Override with `--data-dir` if needed. First run writes a default settings file.

Transports

Configure any combination in `settings.json`. MeshBridge polls on an interval (default 30s) and pushes into the local Base Station.

Manual URL

When you already know the other Base's HTTP address:

```
"manual": [
  { "name": "Site B", "url": "http://192.168.2.10:9091" }
]
```

Ethernet (recommended dual-router setup)

1. Keep Wi-Fi on LAN A (phones / Base A).
2. Plug Ethernet from this Mac into a LAN port on router B (USB-C / Thunderbolt adapter is fine).
3. Confirm the Ethernet interface has a DHCP address on B's subnet (ifconfig or System Settings → Network). Note the device name (en5, en7, ...).
4. Add:

```
"ethernet": [
  { "name": "Office LAN B", "interface": "en5" }
]
```

MeshBridge discovers Base Stations advertising api= on that interface via mDNS. No SSID/password — the cable must already be up.

Secondary Wi-Fi NIC

With a USB Wi-Fi adapter as a second radio, MeshBridge can associate that iface to another SSID and browse mDNS there:

```
"wifi": [
  { "name": "Guest LAN", "ssid": "OtherNet", "password": "...", "interface": "en1" }
]
```

macOS associate uses networksetup; other OSes may need associate done outside MeshBridge first.

Punch (QuakeMesh-style)

For Bases that cannot see each other on a shared LAN, configure a hub host and peer id (see docs/2026-07-14-meshbridge-plan.md). MeshBridge exchanges reachable Base HTTP URLs after punch/relay, then syncs over HTTP like the manual transport.

Remote Users on the dashboard

Open the Base Station dashboard → **Remote Users** (/remote-users). Devices are grouped by origin Base. Live Talk with them from this LAN is not offered; voice notes and channel clips that MeshBridge ingested are.

The page also shows whether MeshBridge has heartbeated recently.

What does not cross

- Live group PTT and private live Talk / SFU Opus
- Automatic mesh ConnectAny to bridged remotes (they stay presence/ingest only)
- GPS trail history replication (same-LAN Multi-Base also does not copy trails today)

MeshSniff

MeshSniff is a companion process that discovers devices on your LAN (and devices MeshBridge already knows) and draws them on a live network map. Click any node for a modal with everything discovered: IPs, MACs, Mesh ID, nickname, GPS, open ports, services, and URLs.

It does **not** replace the Base Station GPS **Map** page, and it does not bridge live Talk.

Running MeshSniff

Start the Base Station (and MeshBridge if you use it), then:

```
source tools/go-env.sh
go run ./meshsniff/cmd/meshsniff
```

Open **http://127.0.0.1:9096**. Settings: ~/Library/Application Support/WalkieTalkie/meshsniff/settings.json on macOS (Linux/Windows use the OS config directory under WalkieTalkie/meshsniff). Build with ./tools/build-macos-meshsniff.sh.

Seed, then discover

1. MeshSniff first asks **MeshBridge** for its inventory (remote Bases and users), placing them on the map even with minimal info.
2. It then loads local Base Station devices and Remote Users.
3. It continuously scans: interface/subnets, ARP cache, TCP service probes, WalkieTalkie mDNS, and optional ICMP when run as root.
4. Open WalkieTalkie ports are probed with GET /sniff (signaling) or GET /api/sniff (Base Station) so MAC addresses can be tied to each device's Mesh ID.

The map updates live as new hosts and details appear.

What you see

- **Networks / subnets** — local interfaces and CIDRs
- **Routers** — default gateway and common .1 / admin-port hosts
- **Hosts** — ARP/TCP-discovered devices
- **WalkieTalkie devices** — phones, Base Stations, Wear that answer identify or appear in mDNS
- **MeshBridge / remote hints** — bridged remotes (dashed) until reached on L3

Status strip shows MeshBridge/Base reachability, ICMP on/off, and last scan duration.

Chapter 10: Wearables

Wear OS and Apple Watch

Wear OS

The Wear OS app is a standalone mesh participant: it embeds the same Go core as the phone, announces itself with platform wear, and joins the LAN over Wi-Fi. Install it separately from the phone app (com.walkietalkie.wear). Current track: 0.2.0.

Swipe sideways between three pages:

- **Talk** — peer count, Base Station hint, and a large **Hold to talk** control.
- **Settings** — nickname for this watch (same idea as the phone).
- **About** — version, device id, a tappable Base Station URL (opens the web UI when a browser is available), and a short explanation of the mesh.

Grant microphone, location, and Bluetooth permissions when asked — the foreground service keeps the mesh and mic alive while your wrist is down.

Voice notes and private-channel UI stay on the phone for now; the watch is for quick group Talk.

The watch and the Base Station must be on the **same Wi-Fi LAN** (same SSID/subnet). Different guest networks or secondary SSIDs will not discover each other over mDNS.

Apple Watch

Apple Watch does not run the Go mesh itself. The watch app is a thin remote control: it uses WatchConnectivity to tell the paired iPhone to start and stop transmitting, and the phone pushes a short status line back (mesh / talking). Keep the iPhone app open (or recently used) on the same Wi-Fi mesh so audio path, BLE, and GPS stay on the phone.

If the watch says the iPhone is unreachable, unlock the phone and open WalkieTalkie once, then try Hold to talk again.

About the Author

Notes

For dual-LAN or remote Base-to-Base sync of devices and voice notes (not live Talk), see the **MeshBridge** chapter. For a LAN discovery network map (MAC ↔ Mesh ID), see **MeshSniff**. Design notes live under docs/2026-07-14-meshbridge-plan.md and docs/2026-07-14-meshsniff-plan.md in the source tree.